Assignment 3: Neural Networks — Feedforward MLP from Scratch on CIFAR-10

Course: CS4101 | CS6103

Name	Roll number
NAME_1	ROLL_1
NAME_2	ROLL_2
NAME_3	ROLL_3

Weights & Biases project: WANDB_LINK

Execution instructions

- **Dependencies:** Python 3.12, `numpy`, `keras` (only to load CIFAR-10; backend `jax`), `wandb`, `pandas`, `matplotlib`, `seaborn`, `scikit-learn`.
  ```
  pip install numpy keras jax wandb pandas matplotlib seaborn scikit-learn jupyter
  wandb login            # once, to sync runs to the online project
  ```
- Run all cells top to bottom (`Kernel → Restart & Run All`). Runs take about 1–2 minutes on a laptop CPU.
- Set `WANDB_MODE=offline` in the environment to run without a W&B account. Offline runs can be uploaded later with `wandb sync wandb/offline-run-*`.

Random seeds

Purpose	Seed
Train/validation subset selection	`np.random.default_rng(42)` (as specified)
Weight initialisation	`np.random.default_rng(INIT_SEED = 0)`
Per-epoch shuffling	`np.random.default_rng(SHUFFLE_SEED = 1)` (one generator per run, so every run sees the same batch order)

```
numpy 2.5.4 | wandb 0.31.0 | wandb mode: offline
```

Dataset and preprocessing

1. Sample 1,000 images per class with `rng = np.random.default_rng(42)`, going through the classes in order 0–9 without resetting the generator. The first 800 of each class go to

training and the remaining 200 to validation. This gives 8,000 training and 2,000 validation images.
2. Scale pixels to `[0, 1]` by dividing by 255. No other normalisation is applied.
3. Flatten each image (all three colour channels) into a 3,072-dimensional vector. Labels stay as integers, and one-hot vectors are built where the loss needs them.
4. Keep the 10,000 official test images untouched until Question 4.

```
train (8000, 3072) val (2000, 3072) test (10000, 3072)
pixel range 0.0 1.0
train per class [800 800 800 800 800 800 800 800 800 800]
val per class   [200 200 200 200 200 200 200 200 200 200]
```

Question 1: Forward propagation

Architecture. The input has 3,072 features, followed by one or two fully connected hidden layers of configurable width with **sigmoid** or **ReLU** activation, and a 10-unit output layer with softmax.

Numerically stable softmax. $\mathrm{softmax}(z)_c = e^{z_c - \max_j z_j} / \sum_k e^{z_k - \max_j z_j}$. Subtracting the row maximum leaves the result unchanged and prevents `exp` from overflowing. Sigmoid is evaluated as $1/(1+e^{-z})$ with z clipped to $[-500, 500]$, which also avoids overflow.

Parameter initialisation (documented choice). Weights are drawn from a zero-mean normal distribution, using the seeded generator `default_rng(INIT_SEED=0)`, and all biases start at zero.

- **He/Kaiming normal** for ReLU hidden layers: $\sigma = \sqrt{2/n_{\text{in}}}$. ReLU zeroes about half of its inputs, and the factor 2 keeps the activation variance roughly constant from layer to layer.
- **Xavier/Glorot normal** for sigmoid hidden layers and for the softmax output layer: $\sigma = \sqrt{2/(n_{\text{in}} + n_{\text{out}})}$. This scale suits activations that are approximately linear around 0.

The weight matrix of layer l has shape (n_{l-1}, n_l) and the bias has shape $(n_l,)$. A batch $X \in \mathbb{R}^{B \times 3072}$ therefore propagates as $Z_l = A_{l-1} W_l + b_l$.

Q1 verification: output shape, valid probabilities, parameter shapes and count

For **3072 → 64 → 10**:

Parameter	Shape	Count
W_1	(3072, 64)	196,608
b_1	(64,)	64
W_2	(64, 10)	640
b_2	(10,)	10

Parameter	Shape	Count
Total		**197,322**

$(3072 \cdot 64 + 64) + (64 \cdot 10 + 10) = 196{,}672 + 650 = 197{,}322$ trainable parameters.

```
[sigmoid] output shape (16, 10); all in [0,1]: True; row sums ~ 1: True
[relu] output shape (16, 10); all in [0,1]: True; row sums ~ 1: True
W1: shape (3072, 64), 196,608 params
b1: shape (64,), 64 params
W2: shape (64, 10), 640 params
b2: shape (10,), 10 params
Total trainable parameters (3072 -> 64 -> 10): 197,322
Two hidden layers 3072 -> 64 -> 64 -> 10: (4, 10) 201,482 params
softmax of extreme logits: [[0.5 0.5 0. ]]
```

Question 2: Backpropagation and training

Loss (mean categorical cross-entropy): $\mathcal{L} = -\frac{1}{B}\sum_i \sum_c y_{ic} \log \hat{p}_{ic}$. For numerical stability the loss is computed from the **logits** with log-softmax, $\log \hat{p} = z - \max z - \log \sum e^{z - \max z}$, rather than as `log(softmax(z))`. Taking the log of a softmax output can produce `log(0) = -inf`.

(a) Gradients. With $\delta_L = \partial \mathcal{L} / \partial Z_L$:

- Softmax combined with cross-entropy gives $\delta_L = (\hat{P} - Y)/B$.
- For each layer l: $\nabla W_l = A_{l-1}^\top \delta_l$, $\nabla b_l = \sum_i \delta_{l,i}$, and $\delta_{l-1} = (\delta_l W_l^\top) \odot f'(Z_{l-1})$.
- The activation derivatives are $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ and $\mathrm{ReLU}'(z) = 1[z > 0]$.

The analytic gradients are checked against central finite differences below.

```
sigmoid hidden=(8,): max relative error = 2.80e-08
sigmoid hidden=(8, 6): max relative error = 1.59e-06
relu    hidden=(8,): max relative error = 7.64e-08
relu    hidden=(8, 6): max relative error = 2.72e-07
```

(b) Optimisers.

- Mini-batch SGD: $\theta_t = \theta_{t-1} - \eta \nabla_\theta \mathcal{L}_{\mathcal{B}_t}(\theta_{t-1})$.
- Momentum: $v_t = 0.9\, v_{t-1} + \nabla_\theta \mathcal{L}_{\mathcal{B}_t}(\theta_{t-1})$, then $\theta_t = \theta_{t-1} - \eta v_t$. Every velocity starts at zero when a run begins and is kept across batches and epochs.

(c) Before every epoch the training set is shuffled with a per-run generator `default_rng(SHUFFLE_SEED=1)`. Weights are initialised with `default_rng(INIT_SEED=0)`.

(d) After every epoch the model is evaluated on the **complete** training subset (8,000 images) and validation subset (2,000 images), and loss and accuracy are recorded for both.

Checkpointing. A run's checkpoint is its parameters at the first epoch that reaches the run's highest validation accuracy. Only a *strictly* higher accuracy replaces the stored copy, so the earliest

checkpoint is the one kept.

Training time. Measured with `time.perf_counter()` and summed over the mini-batch steps only (forward, backward and parameter update). The per-epoch full-set evaluations, checkpoint copies and W&B logging are excluded, so the number reflects the cost of training itself.

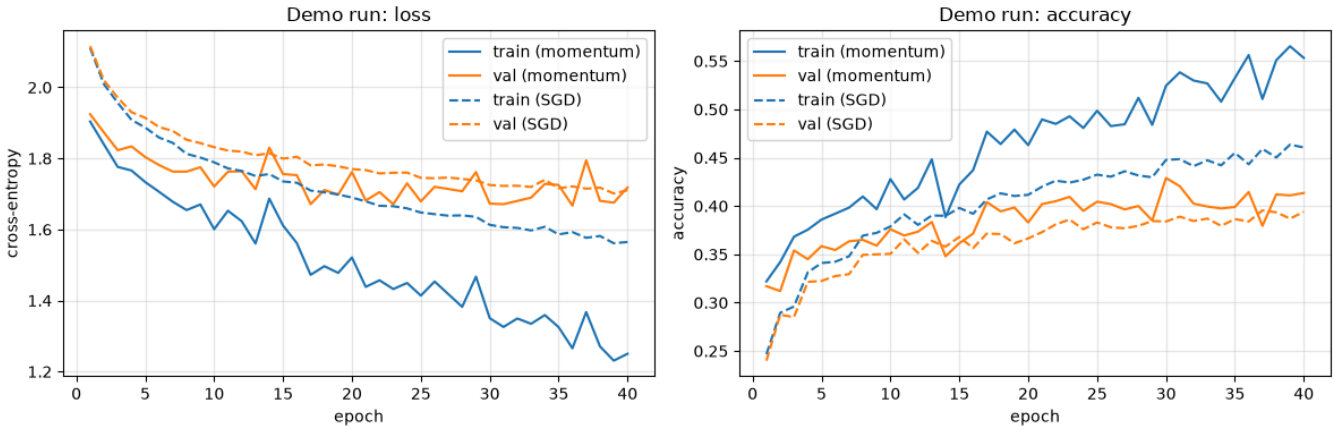
Everything is driven by a `config` dict, so experiments never require changes to the training loop.

(e) A complete training run

Chosen setup: architecture 3072 → 64 → 10 with ReLU, momentum optimiser ($\beta = 0.9$), learning rate $\eta = 0.005$, batch size 64, 40 epochs.

The learning rate was picked in a short preliminary sweep over {0.01, 0.005, 0.001} that looked at validation accuracy only. At 0.01 the ReLU validation loss was noisy, and at 0.001 the sigmoid networks learned too slowly to converge within 40 epochs. A batch size of 64 gives 125 updates per epoch.

For comparison, the same network is also trained with plain mini-batch SGD at the same learning rate.



Hyperparameter tuning

- Hidden widths: **64, 128 and 256**, each tried with **sigmoid** and **ReLU**, giving 6 configurations.
- Fixed for every run: one hidden layer, momentum optimiser ($\beta = 0.9$), $\eta = 0.005$, batch size 64, 40 epochs, initialisation seed 0, shuffle seed 1.
- Selection uses the validation set only. The test set is not touched here.

Question 3: Present and interpret the results

(a) Results table

How training time was measured: wall-clock `time.perf_counter()` summed over the mini-batch forward, backward and update steps of all 40 epochs. Per-epoch evaluation and logging are excluded. All runs were on the same CPU with NumPy's default multithreaded BLAS, so absolute times vary between machines but the relative times are comparable.

The same table is logged to W&B as `results_table`.

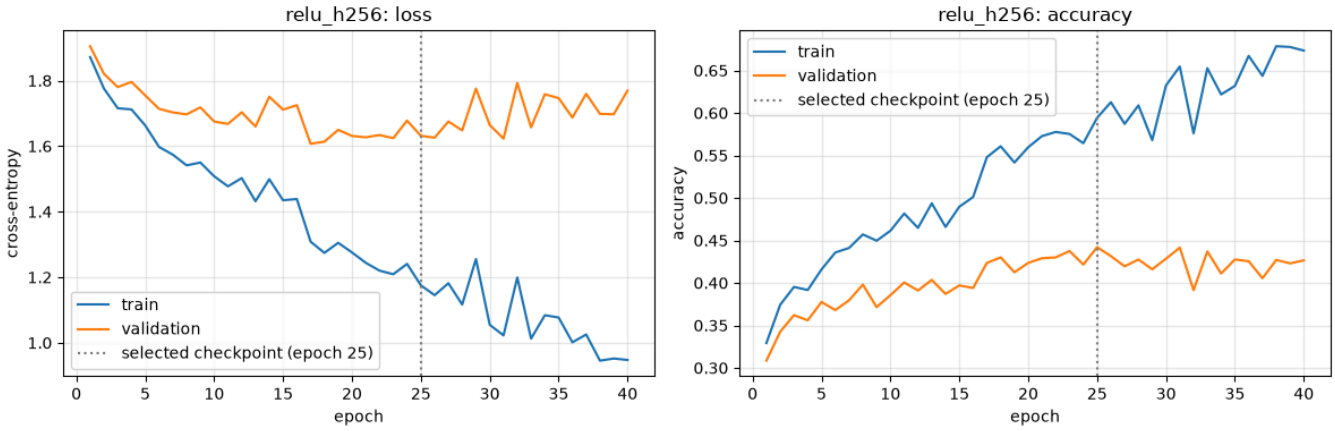
run	configuration	params	best_val_acc	best_epoch	final_train_acc	final_val_acc	train_time_s
0	3072-64-10 sigmoid	197,322	0.4100	39	0.4880	0.4080	2.02
1	3072-64-10 relu	197,322	0.4290	30	0.5533	0.4135	1.92
2	3072-128-10 sigmoid	394,634	0.4145	39	0.4990	0.4100	3.60
3	3072-128-10 relu	394,634	0.4345	38	0.6272	0.4110	3.40
4	3072-256-10 sigmoid	789,258	0.4135	40	0.4981	0.4135	7.54
5	3072-256-10 relu	789,258	0.4420	25	0.6734	0.4265	7.08

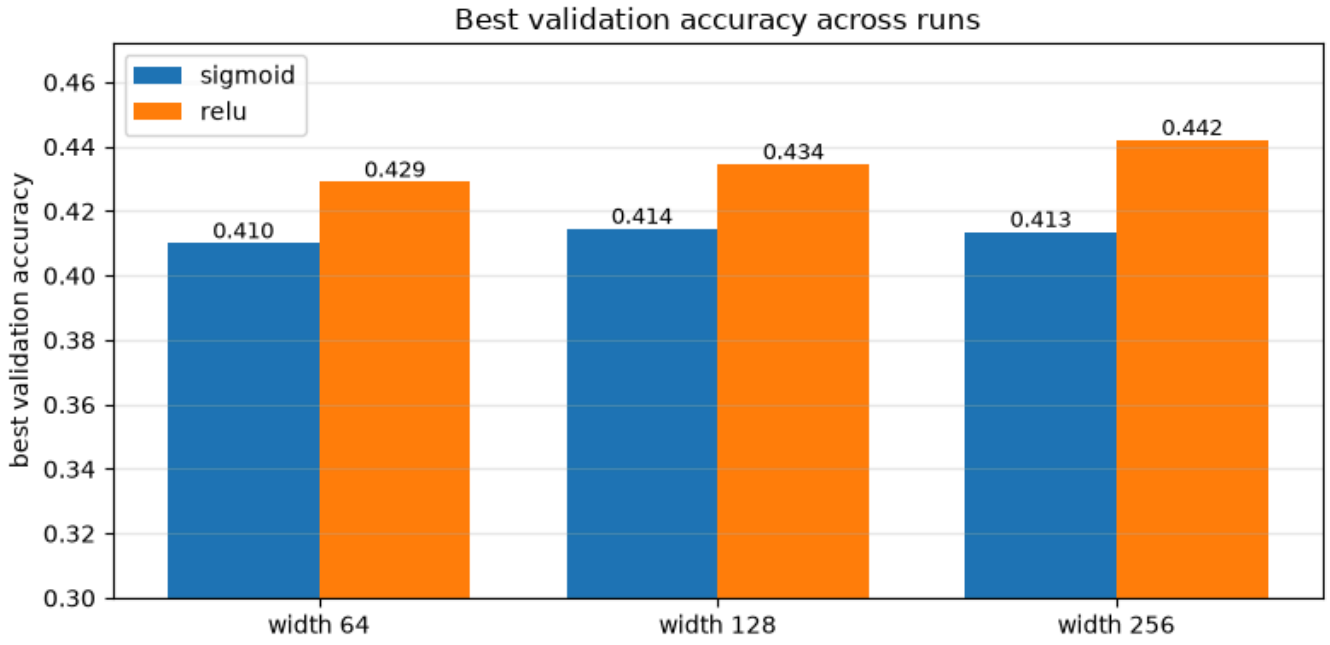
(b) Model selection

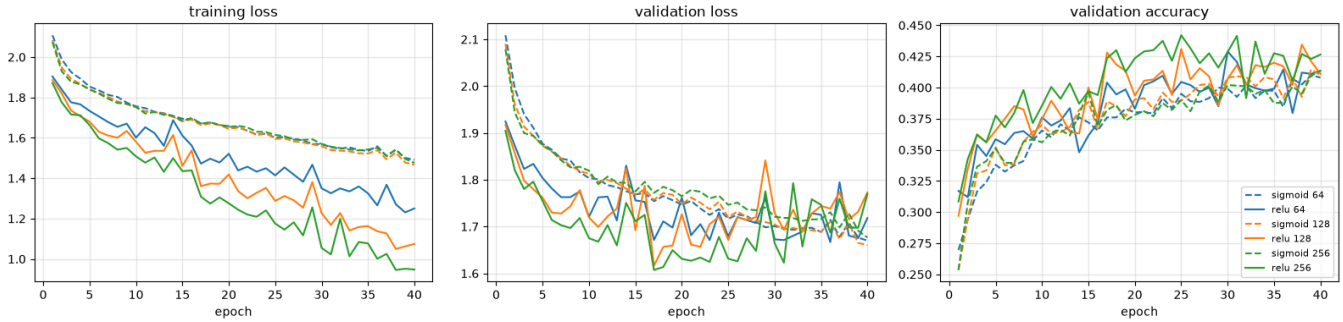
The selection rule is the highest best-validation accuracy. Within a run, the earliest epoch that reached it is kept (enforced during training). Ties between runs go to the run with fewer parameters, and then to the earlier run.

```
Selected run 5: relu_h256 | best val acc 0.4420 at epoch 25 | 789,258 params
```

(c) Learning curves of the selected run and comparison across runs







(d) Discussion: activation function, width, under/overfitting

	width 64	width 128	width 256
sigmoid: best val acc (epoch)	0.4100 (39)	0.4145 (39)	0.4135 (40)
ReLU: best val acc (epoch)	0.4290 (30)	0.4345 (38)	**0.4420 (25)**
sigmoid: final train – val acc	8.0 pp	8.9 pp	8.5 pp
ReLU: final train – val acc	14.0 pp	21.6 pp	24.7 pp

Activation function. ReLU beats sigmoid at every width, by about 2–3 percentage points, and it also learns faster: after the first epoch ReLU validation accuracy is 0.30–0.32 versus 0.25–0.27 for sigmoid. The sigmoid derivative is at most 0.25 and becomes very small when units saturate, so gradients reaching W_1 are damped. ReLU passes the gradient through unchanged for every active unit.

Hidden-layer width. Width barely matters for sigmoid: the three runs fall within 0.5 pp of each other, which is inside the epoch-to-epoch noise. For ReLU, accuracy rises steadily with width (0.429

→ 0.435 → 0.442). The extra capacity costs about 4× the parameters (197k → 789k) and about 3.7× the training time (1.9 s → 7.0 s). Almost all of the parameters sit in the 3072 × width input layer.

Underfitting or overfitting?

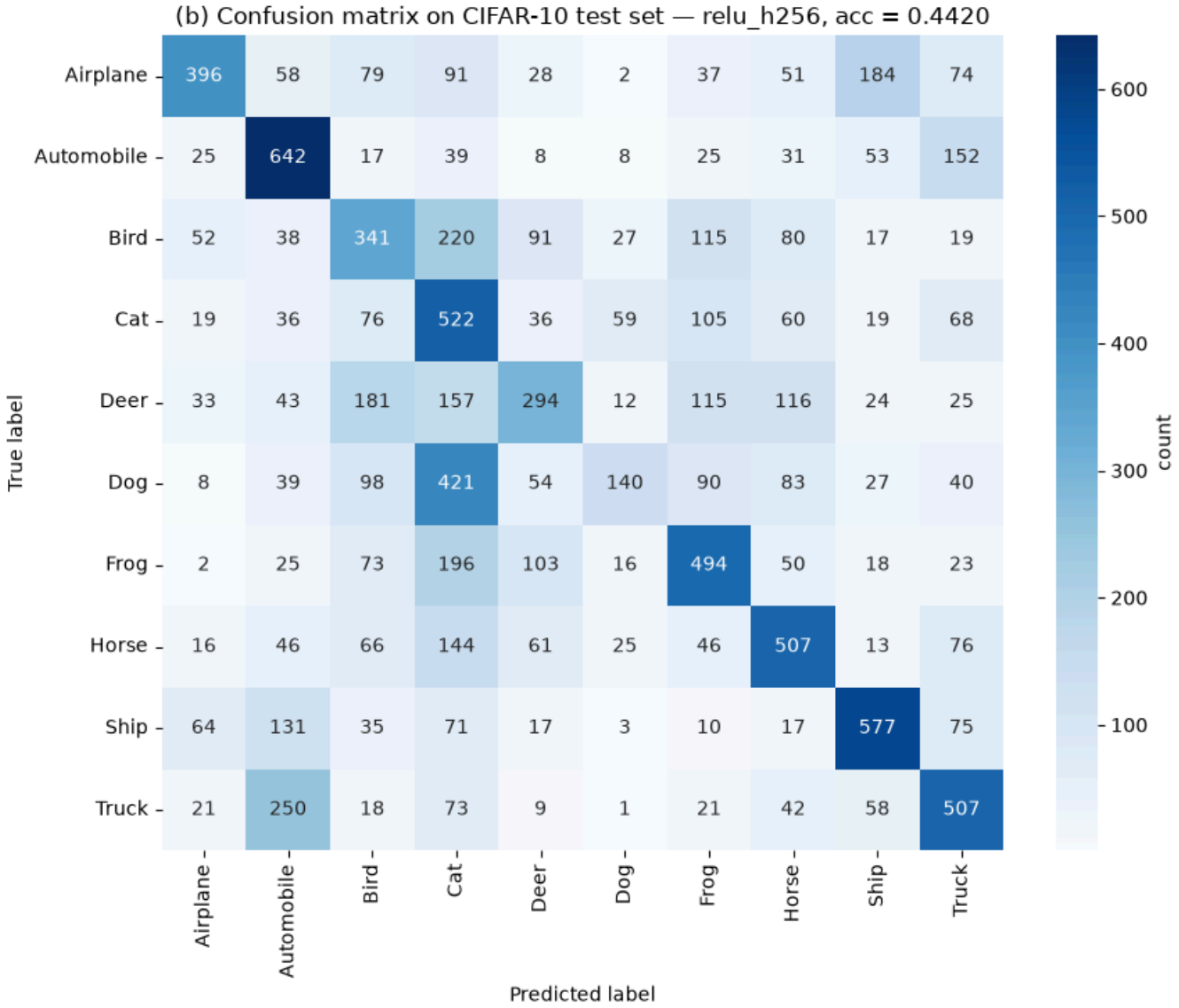
- *Sigmoid runs mostly underfit or are under-trained.* Training and validation losses are still falling together at epoch 40, and the best checkpoints come from epochs 39–40. The train–validation gap stays small (about 8–9 pp). More epochs or a larger learning rate would probably still help.
- *ReLU runs overfit, and wider networks overfit more.* Training loss keeps falling (to 0.95 for width 256), while validation loss bottoms out around epochs 17–25 (≈1.61 for width 256) and then rises to ≈1.77. The train–validation accuracy gap widens from 14 pp at width 64 to 25 pp at width 256. This is why the selected checkpoint comes from epoch 25 and not from the end of training. Early stopping through checkpoint selection acts as the regulariser here. Weight decay, dropout or data augmentation would be the next steps.
- Even so, training accuracy remains modest for every model (49–67%). Much of the error is bias from the representation (see Q4c), not only variance.

Noise caveat. With momentum and batches of 64, validation accuracy fluctuates by about ±1–2 pp from one epoch to the next, and each configuration was run with one seed. Differences smaller than about 1 pp, such as between the sigmoid widths, should therefore not be over-interpreted.

Question 4: Evaluate the selected model

The checkpoint is restored from `checkpoints/best_model.npz` into a freshly constructed network and evaluated **once** on the 10,000 official test images. The model and its hyperparameters were fixed before this step and are not changed in response to the test results.

```
Selected model: relu_h256 (epoch 25)
(a) Test accuracy: 0.4420   (test loss 1.6442)
```



	precision	recall	f1-score	support
Airplane	0.623	0.396	0.484	1000
Automobile	0.491	0.642	0.556	1000
Bird	0.347	0.341	0.344	1000
Cat	0.270	0.522	0.356	1000
Deer	0.419	0.294	0.346	1000
Dog	0.478	0.140	0.217	1000
Frog	0.467	0.494	0.480	1000
Horse	0.489	0.507	0.498	1000
Ship	0.583	0.577	0.580	1000
Truck	0.479	0.507	0.492	1000
accuracy			0.442	10000
macro avg	0.464	0.442	0.435	10000
weighted avg	0.464	0.442	0.435	10000

(c) Limitations of flattened pixels with a fully connected network

The selected model reaches **44.2% test accuracy**, which matches its validation accuracy, so model selection did not overfit the validation set. This is far above chance (10%) but well below simple CNNs (≥ 70–80%). The confusion matrix shows where the errors come from:

- **No spatial structure.** Flattening discards the 2-D neighbourhood of pixels. To the network, pixel (5, 5) and pixel (5, 6) are unrelated inputs, so it has to learn edges, textures and shapes from scratch without any built-in notion of locality.
- **No translation invariance or weight sharing.** Each first-layer weight is tied to one absolute pixel position. An object that shifts by a few pixels produces a completely different input vector, and the same pattern must be learned again at every location. This wastes parameters: 786k of the 789k weights sit in the first layer, and they still overfit 8,000 images, as shown by the train–validation gap in Q3.
- **Reliance on global colour and background.** Without local feature detectors the model falls back on coarse cues such as overall colour and layout. The typical confusions follow those cues: airplane → ship (184, blue sky versus blue sea), automobile ↔ truck (152 / 250), deer → bird / frog (181 / 115, green and brown backgrounds), and especially **dog → cat (421)**. Dog recall is only 14%, because the two classes differ in fine shape detail, not in colour statistics.
- **Sensitivity to nuisance factors.** Changes in lighting, scale, viewpoint or position move the raw pixel vector a long way, so classes are not close to separable in pixel space. The network has no prior to undo these changes, and with only 800 images per class and no augmentation it cannot learn that invariance from data either.
- **Poor scaling.** Larger images or deeper networks make the parameter count of fully connected layers explode. Convolutional layers, which use local receptive fields, shared weights and pooling, address all of the points above, but they are outside the scope of this assignment.